

Laporan Hands-on 4

Threads and concurrency dibahas melalui dua pendekatan, yaitu simulasi concurrency pada Bash dan implementasi multithreading nyata menggunakan **POSIX threads (pthread)** pada bahasa C. Rangkaian bahasan ini memperlihatkan perkembangan konsep dari eksekusi independen, responsivitas, race condition, mutual exclusion, hingga blocking behavior. Intinya, concurrency bukan hanya soal mempercepat eksekusi, tetapi juga soal **correctness, sinkronisasi, dan pengelolaan shared resource**.

Section 1 - Sequential vs Concurrent Execution pada Bash

Dua model eksekusi paling dasar yang dibandingkan di sini adalah **sequential execution** dan **concurrent execution**. Pada mode sequential, setiap task harus selesai terlebih dahulu sebelum task berikutnya dimulai. Sebaliknya, pada mode concurrent, beberapa task dijalankan sebagai background process menggunakan operator **&**, sehingga waktu tunggu dapat saling overlap. Fokus utamanya adalah menunjukkan bahwa concurrency memberi manfaat paling jelas ketika task-task yang dikerjakan **independen** dan didominasi oleh waktu tunggu.

Implementasi lab menggunakan tiga task sederhana yang masing-masing mencetak status mulai, menunggu selama 3 detik dengan **sleep**, lalu mencetak status selesai. Ketika ketiga task dijalankan secara sequential, total waktu eksekusi menjadi kira-kira **9 detik** karena tiga waktu tunggu dieksekusi satu per satu. Namun ketika ketiga task dijalankan secara concurrent sebagai background process, ketiganya dapat menunggu secara bersamaan sehingga total waktu berkurang menjadi sekitar **3 detik**. Hasil ini menunjukkan bahwa concurrency dapat meningkatkan efisiensi waktu tanpa harus mengubah pekerjaan inti masing-masing task.

Dari sudut pandang sistem operasi, eksperimen ini memperlihatkan bahwa scheduler dapat mengatur lebih dari satu alur eksekusi dalam rentang waktu yang sama. Walaupun Bash tidak menciptakan thread di dalam satu process seperti pthread, penggunaan background process sudah cukup untuk menunjukkan efek **overlapping progress**. Output juga dapat muncul dalam urutan yang berbeda-beda, sehingga praktikum ini sekaligus mengenalkan sifat **nondeterministic ordering** pada sistem concurrent.

Ringkasan Eksperimen

Komponen	Informasi
File praktikum	lab1.sh
Mekanisme utama	Background process dengan & dan sinkronisasi akhir dengan wait
Jumlah task	3 task (A, B, C)
Durasi tunggu per task	3 detik
Total waktu sequential	Sekitar 9 detik

Komponen	Informasi
Total waktu concurrent	Sekitar 3 detik
Konsep utama	Overlap waiting time pada task independen

Kesimpulan Section 1

Concurrency dapat mengurangi total waktu eksekusi ketika beberapa task tidak saling bergantung. Dengan menjalankan task sebagai background process, waktu tunggu yang sebelumnya dijalankan secara berurutan dapat ditumpuk secara bersamaan. Hasil perbandingan tersebut menjadi fondasi penting untuk memahami bahwa concurrency sangat efektif untuk workload yang mengandung banyak aktivitas menunggu.

Section 2 - Foreground dan Background Task pada Bash

Sebuah program tetap dapat responsif terhadap input pengguna walaupun ada aktivitas lain yang berjalan di background. Satu loop dijalankan sebagai background task yang secara periodik mencetak pesan ke terminal, sedangkan loop lain tetap berada di foreground untuk menerima input dari pengguna. Dengan cara ini, terlihat bahwa concurrency tidak hanya berhubungan dengan kecepatan, tetapi juga dengan **responsiveness**.

Task background pada lab ini mencetak status periodik setiap 5 detik, sedangkan task foreground membaca input dengan `read -p` dan segera menampilkan kembali perintah yang diketik pengguna. Ketika program dijalankan, pengguna tetap dapat memasukkan perintah kapan pun tanpa perlu menunggu task background selesai. Hal ini menyerupai perilaku aplikasi nyata seperti text editor yang melakukan autosave sambil tetap menerima input keyboard, atau browser yang memuat resource sambil tetap dapat di-scroll dan diklik.

Dari hasil pengamatan, muncul satu fenomena penting lain, yaitu **interferensi output pada shared terminal**. Karena foreground task dan background task menulis ke layar yang sama, tampilan terminal dapat terlihat berantakan ketika kedua alur menulis hampir bersamaan. Dengan demikian, lab ini memperlihatkan dua sisi concurrency sekaligus: concurrency meningkatkan kenyamanan pengguna, tetapi juga dapat menimbulkan kebutuhan akan koordinasi ketika dua alur mengakses resource yang sama.

Ringkasan Eksperimen

Komponen	Informasi
File praktikum	lab2.sh
Foreground task	Menerima input pengguna dengan <code>read</code>
Background task	Mencetak pesan periodik setiap 5 detik
Shared resource	Terminal output
Manfaat utama	Responsivitas antarmuka tetap terjaga
Risiko utama	Output saling menimpa atau terlihat tidak rapi
Konsep utama	Foreground/background concurrency dan shared output

Kesimpulan Section 2

Concurrency sangat penting untuk menjaga responsivitas aplikasi. Walaupun satu task terus berjalan di background, pengguna masih dapat berinteraksi dengan program tanpa jeda berarti. Pada saat yang sama, eksperimen ini memperkenalkan bahwa shared resource sederhana seperti terminal pun dapat menjadi sumber gangguan jika banyak alur eksekusi menulis secara bersamaan.

Section 3 - Race Condition pada Shared File

Bentuk awal **race condition** terlihat melalui shared resource yang sangat sederhana, yaitu file `counter.txt`. Tiga background process menjalankan fungsi increment yang masing-masing melakukan 100 kali update ke counter yang sama. Secara matematis, hasil akhir seharusnya bernilai **300**, tetapi pada praktiknya hasil aktual sering kali lebih kecil. Perbedaan inilah yang menjadi bukti bahwa concurrency dapat merusak correctness ketika shared resource diakses tanpa sinkronisasi.

Masalah utamanya terletak pada pola operasi **read-modify-write** yang tidak atomik. Setiap process membaca nilai lama dari file, menambahkan satu, lalu menulis kembali hasilnya. Jika dua process membaca nilai yang sama sebelum salah satunya sempat menulis hasil baru, maka salah satu increment akan hilang. Sebagai contoh, dua process dapat sama-sama membaca nilai 10, sama-sama menghitung 11, lalu keduanya menulis 11. Secara logika ada dua increment, tetapi hasil tersimpan hanya bertambah satu.

Dari sudut pandang sistem operasi, eksperimen ini sangat penting karena menunjukkan bahwa kesalahan concurrent program sering kali bukan berasal dari syntax error atau logika lokal yang salah, melainkan dari **timing**. Program terlihat benar bila dibaca secara sequential, tetapi ketika beberapa alur berjalan bersamaan, urutan interleaving instruksi dapat mengubah hasil akhir. Inilah alasan mengapa concurrency membutuhkan mekanisme proteksi khusus terhadap shared data.

Ringkasan Eksperimen

Komponen	Informasi
File praktikum	lab3.sh
Shared resource	counter.txt
Jumlah process	3 background process
Increment per process	100 kali
Nilai yang diharapkan	300
Nilai yang sering muncul	Kurang dari 300
Konsep utama	Race condition dan lost update

Kesimpulan Section 3

Akses concurrent terhadap shared file tanpa sinkronisasi dapat menghasilkan nilai akhir yang salah. Walaupun setiap process menjalankan logika increment yang tampak benar, hasil global tetap dapat rusak karena dua update saling menimpa. Kondisi tersebut

memperjelas bahwa concurrency tanpa proteksi dapat meningkatkan kecepatan, tetapi sekaligus mengorbankan correctness.

Section 4 - Perbaikan Race Condition dengan Lock

Kelanjutan dari pengujian sebelumnya menambahkan mekanisme sinkronisasi sederhana berupa **lock directory**. Ide dasarnya adalah hanya process yang berhasil membuat direktori lockdir yang diizinkan masuk ke critical section, sedangkan process lain harus menunggu sampai lock dilepas. Dengan pendekatan ini, operasi baca-tambah-tulis terhadap counter.txt tidak lagi dapat tumpang tindih.

Hasil eksperimen menunjukkan perubahan yang sangat jelas. Nilai counter yang sebelumnya sering kurang dari 300 kini menjadi **konsisten 300** pada setiap run normal. Ini membuktikan bahwa mutual exclusion berhasil mengubah operasi yang tadinya rawan interleaving menjadi update yang aman. Dari sisi correctness, program menjadi lebih stabil dan hasilnya sesuai dengan ekspektasi matematis.

Namun, pembahasan ini juga memperlihatkan trade-off penting dalam concurrent system design. Ketika hanya satu process yang boleh masuk ke critical section pada satu waktu, maka sebagian alur program menjadi **serialized**. Artinya, sinkronisasi memang memperbaiki kebenaran hasil, tetapi dapat mengurangi paralelisme efektif. Dengan kata lain, concurrency yang benar sering kali membutuhkan kompromi antara **speed** dan **safety**.

Ringkasan Eksperimen

Komponen	Informasi
File praktikum	lab4.sh
Mekanisme lock	mkdir lockdir sebagai lock sederhana
Shared resource	counter.txt
Jumlah process	3 background process
Nilai yang diharapkan	300
Nilai aktual	Konsisten 300
Konsep utama	Mutual exclusion dan biaya sinkronisasi

Kesimpulan Section 4

Race condition dapat dihilangkan dengan mutual exclusion. Lock memastikan hanya satu process yang mengakses shared resource pada satu waktu sehingga lost update tidak lagi terjadi. Hasil ini juga menegaskan bahwa sinkronisasi bukan membuat program lebih “ajaib”, melainkan membuat program concurrent menjadi **benar**.

Section 5 - Simulasi Blocking pada Bash

Konsep **blocking is local** tampak ketika satu task berhenti menunggu tetapi task lain masih tetap dapat berjalan. Task A hanya melakukan sleep 5 lalu mencetak pesan selesai,

sedangkan Task B terus bekerja dan menampilkan progres langkah demi langkah. Keduanya dijalankan secara concurrent sebagai background process.

Ketika program dijalankan, mahasiswa akan melihat bahwa Task B tetap mencetak progress setiap detik walaupun Task A sedang “diam” menunggu selama lima detik. Hal ini menunjukkan bahwa blocking pada satu task tidak serta-merta menghentikan semua aktivitas lain dalam sistem. Inilah salah satu alasan mengapa concurrency sangat penting dalam sistem operasi: waktu tunggu dari satu task dapat dimanfaatkan untuk mengerjakan task lain.

Secara konseptual, gambaran ini menjadi jembatan menuju pemahaman thread blocking pada tingkat yang lebih nyata. Dalam sistem operasi modern, ketika sebuah alur menunggu I/O atau timer, scheduler dapat menjalankan alur lain yang siap. Dengan demikian, CPU tidak terbuang hanya untuk menunggu satu aktivitas selesai. Ide tersebut tampak dalam bentuk yang sederhana dan mudah diamati.

Ringkasan Eksperimen

Komponen	Informasi
File praktikum	lab5.sh
Task A	Menunggu 5 detik lalu selesai
Task B	Mencetak 5 langkah kerja, masing-masing jeda 1 detik
Hubungan task	Independen
Fenomena utama	Task B tetap aktif saat Task A diblok
Konsep utama	Local blocking dan overlap antara waiting dengan useful work

Kesimpulan Section 5

Blocking bersifat lokal pada task yang memanggil operasi blocking. Walaupun satu task berhenti karena menunggu, task lain tetap dapat berjalan dan menghasilkan progres. Dari sini terlihat bahwa concurrency membantu sistem tetap produktif meskipun sebagian alurnya sedang berada dalam keadaan menunggu.

Section 6 - Sequential vs Multithread Execution pada C

Pembahasan concurrency lalu berpindah dari Bash ke **POSIX threads** dalam bahasa C. Jika pada bagian Bash concurrency disimulasikan dengan background process, pada tahap ini concurrency benar-benar terjadi di dalam satu process yang memiliki beberapa thread. Tiga thread dibuat hampir bersamaan, masing-masing menjalankan fungsi task, lalu main thread menunggu semuanya selesai dengan `pthread_join`.

Hasil yang diamati serupa dengan eksperimen Bash sebelumnya: ketiga task mulai hampir bersamaan dan selesai setelah kira-kira 3 detik, bukan 9 detik. Namun perbedaannya adalah semua alur eksekusi kini berada di dalam satu address space yang sama. Ini sangat penting karena thread jauh lebih ringan daripada process, tetapi juga berarti mereka lebih

mudah berbagi data, yang nanti menjadi sumber masalah race condition pada lab berikutnya.

Output yang muncul juga memperlihatkan sifat nondeterministic ordering. Pesan dari Task A, B, dan C dapat muncul dalam urutan berbeda-beda pada setiap run. Dengan demikian, eksperimen ini menegaskan kembali bahwa multithreading meningkatkan concurrency, tetapi tidak pernah menjamin urutan eksekusi kecuali urutan itu dipaksakan dengan sinkronisasi eksplisit.

Ringkasan Eksperimen

Komponen	Informasi
File praktikum	lab6.c
Library utama	pthread
Jumlah thread	3 worker thread
Durasi kerja per thread	sleep(3)
Sinkronisasi akhir	pthread_join
Total waktu	Sekitar 3 detik
Konsep utama	True multithreading dalam satu process

Kesimpulan Section 6

Thread memungkinkan beberapa jalur eksekusi berjalan di dalam satu process. Ketika task-task independen dijalankan sebagai thread, total waktu penyelesaian dapat dipersingkat melalui overlap. Hasil ini menjadi fondasi penting sebelum masuk ke topik shared memory dan sinkronisasi thread.

Section 7 - Background Thread di Dalam Satu Process

Pola aplikasi yang umum digunakan di dunia nyata terlihat ketika satu thread melakukan pekerjaan latar belakang secara periodik sementara main thread tetap menjalankan aktivitas utama. Pada skenario yang diuji, background thread mencetak pesan setiap 3 detik, sedangkan main thread bekerja langkah demi langkah setiap 1 detik. Tujuannya adalah membuktikan bahwa satu process dapat memiliki beberapa tanggung jawab berbeda yang dijalankan oleh thread-thread terpisah.

Berbeda dengan background process di Bash, seluruh aktivitas pada lab ini hidup di dalam satu process. Artinya, ketika fungsi main selesai dan process berakhir, background thread juga ikut berhenti. Ini menegaskan bahwa thread tidak berdiri sendiri seperti process, melainkan bergantung pada lifecycle process induknya. Dari sudut pandang sistem operasi, konsep ini sangat penting karena menunjukkan hubungan antara **process ownership** dan **thread execution path**.

Manfaat concurrency dari sisi desain perangkat lunak juga terlihat jelas. Aplikasi dapat memisahkan pekerjaan user-facing dan background maintenance menjadi thread yang berbeda tanpa harus menciptakan process baru. Susunan seperti ini menjadi pengantar

yang baik sebelum masuk ke kasus yang lebih sulit, yaitu saat thread-thread di dalam satu process mulai berbagi data yang sama.

Ringkasan Eksperimen

Komponen	Informasi
File praktikum	lab7.c
Thread utama	Mencetak progres kerja setiap 1 detik
Background thread	Mencetak pesan periodik setiap 3 detik
Lingkup eksekusi	Satu process yang sama
Fenomena utama	Background thread berhenti saat process berakhir
Konsep utama	Background thread dan lifecycle thread di dalam process

Kesimpulan Section 7

Thread memungkinkan satu process membagi tanggung jawab ke dalam beberapa jalur eksekusi. Pendekatan ini membuat program lebih modular dan responsif. Pada saat yang sama, eksperimen ini menegaskan bahwa semua thread tetap terikat pada lifecycle process yang menaunginya.

Section 8 - Race Condition pada Shared Variable di C

Race condition kemudian muncul dalam bentuk yang lebih nyata daripada shared file, yaitu **shared memory** di antara thread-thread dalam satu process. Variabel global counter diincrement oleh tiga thread, masing-masing sebanyak 100000 kali. Secara matematis, hasil akhirnya harus **300000**, tetapi pada praktiknya nilai aktual sering lebih kecil. Hal ini menunjukkan bahwa masalah race condition menjadi lebih mudah muncul ketika thread benar-benar berbagi address space.

Penyebab utama masalah ini adalah operasi counter++ bukanlah operasi atomik. Walaupun tampak seperti satu perintah sederhana, secara internal ia melibatkan tahapan membaca nilai saat ini, menambah satu, lalu menulis kembali hasil baru. Jika dua thread melakukan rangkaian langkah ini secara tumpang tindih, maka satu update dapat menimpa update lainnya dan menghasilkan lost update. Inilah bentuk klasik race condition pada shared variable.

Sisi gelap multithreading terlihat jelas pada bagian ini. Pada eksperimen sebelumnya, thread memberi keuntungan besar dalam hal concurrency dan responsivitas. Namun ketika thread mulai berbagi data tanpa sinkronisasi, correctness program tidak lagi dapat dijamin. Dengan demikian, bagian ini menyiapkan landasan konseptual untuk memahami pentingnya mutex pada lab berikutnya.

Ringkasan Eksperimen

Komponen	Informasi
----------	-----------

Komponen	Informasi
File praktikum	lab8.c
Shared resource	Variabel global counter
Jumlah thread	3 thread
Increment per thread	100000 kali
Nilai yang diharapkan	300000
Nilai aktual	Sering lebih kecil dari 300000
Konsep utama	Race condition pada shared memory

Kesimpulan Section 8

Shared memory membuat komunikasi antarthread sangat mudah, tetapi juga membuat kesalahan sinkronisasi jauh lebih berbahaya. Tanpa proteksi, operasi sederhana `counter++` pun dapat menghasilkan nilai akhir yang salah. Hal tersebut menegaskan bahwa multithreading tanpa synchronization tidak cukup untuk membangun program yang benar.

Section 9 - Perbaiki Race Condition dengan Mutex

Perbaikan terhadap pengujian sebelumnya dilakukan dengan menambahkan **mutex** untuk melindungi critical section. Setiap kali thread ingin mengupdate counter, thread harus memanggil `pthread_mutex_lock`, melakukan increment, lalu melepaskan lock dengan `pthread_mutex_unlock`. Dengan begitu, hanya satu thread yang boleh masuk ke bagian kode yang mengakses shared variable pada satu waktu.

Hasil praktikum memperlihatkan bahwa nilai counter kini menjadi **konsisten 300000**. Tidak seperti lab sebelumnya, hasil ini stabil pada run berulang karena mutex memaksa operasi read-modify-write berjalan seolah-olah atomik dari sudut pandang thread lain. Inilah bentuk paling langsung dari mutual exclusion pada shared memory di C.

Walaupun correctness berhasil dipulihkan, pembahasan ini juga menekankan trade-off yang sama seperti sinkronisasi pada Bash. Ketika setiap increment harus menunggu lock, sebagian concurrency efektif berkurang karena update menjadi serialized. Dengan demikian, mahasiswa dapat melihat bahwa desain sinkronisasi yang baik selalu berusaha menjaga keseimbangan antara **keamanan data** dan **kinerja**.

Ringkasan Eksperimen

Komponen	Informasi
File praktikum	lab9.c
Mekanisme proteksi	<code>pthread_mutex_t lock</code>
Shared resource	Variabel global counter
Jumlah thread	3 thread
Nilai yang diharapkan	300000
Nilai aktual	Konsisten 300000

Komponen	Informasi
Konsep utama	Mutual exclusion dengan mutex

Kesimpulan Section 9

Mutex merupakan solusi efektif untuk menghilangkan race condition pada shared variable. Dengan melindungi critical section, program concurrent kembali menghasilkan nilai yang benar dan konsisten. Hasil tersebut menegaskan bahwa sinkronisasi thread bukan fitur tambahan, melainkan kebutuhan dasar ketika ada shared mutable data.

Section 10 - Blocking Threads pada C

Blocking behavior pada thread memiliki prinsip yang sama seperti blocking pada process: **blocking bersifat lokal terhadap thread yang menunggu**. Pada skenario penutup, Thread A mensimulasikan operasi I/O lambat dengan `sleep(6)`, sedangkan Thread B terus mencetak langkah kerja setiap detik. Keduanya dibuat di dalam satu process dan disinkronkan di akhir dengan `pthread_join`.

Hasil pengamatan memperlihatkan bahwa walaupun Thread A sedang menunggu, Thread B tetap berjalan normal dan menyelesaikan progresnya. Setelah sekitar enam detik, Thread A baru melaporkan bahwa operasi I/O selesai, lalu main thread menunggu sampai kedua thread benar-benar berakhir. Dengan demikian, blocking pada satu thread tidak membuat seluruh process ikut berhenti selama masih ada thread lain yang runnable.

Dari sudut pandang sistem operasi, pembahasan ini memperjelas salah satu manfaat terbesar multithreading, yaitu **responsivitas selama operasi lambat berlangsung**. Waktu tunggu yang dimiliki satu thread dapat diisi oleh thread lain dengan pekerjaan berguna. Hal ini sangat penting dalam aplikasi yang berinteraksi dengan I/O, jaringan, timer, atau input pengguna.

Ringkasan Eksperimen

Komponen	Informasi
File praktikum	lab10.c
Thread A	Simulasi I/O dengan <code>sleep(6)</code>
Thread B	Mencetak 6 langkah kerja dengan jeda 1 detik
Sinkronisasi akhir	<code>pthread_join</code> pada kedua thread
Fenomena utama	Thread B tetap aktif saat Thread A diblok
Konsep utama	Blocking pada level thread, bukan seluruh process

Kesimpulan Section 10

Thread memungkinkan sebuah process tetap responsif walaupun salah satu jalur eksekusinya sedang diblok. Inilah alasan mengapa thread sangat berguna untuk aplikasi dengan operasi lambat atau asynchronous. Selama data yang dibagikan dikelola dengan benar, multithreading dapat meningkatkan produktivitas sistem tanpa membuat seluruh aplikasi ikut berhenti saat satu operasi menunggu.

Penutup

Berdasarkan seluruh rangkaian praktikum ini, dapat dilihat bahwa pembelajaran concurrency berkembang secara bertahap dari konsep paling dasar hingga masalah sinkronisasi yang lebih serius. Bagian Bash menunjukkan bahwa concurrency dapat mempercepat task independen, menjaga responsivitas, dan memperlihatkan bagaimana blocking pada satu alur tidak menghentikan alur lain. Setelah itu, bagian pthread menunjukkan bentuk concurrency yang lebih nyata di dalam satu process, termasuk keuntungan multithreading sekaligus risiko race condition pada shared memory.

Secara keseluruhan, rangkaian praktikum ini menegaskan bahwa concurrency bukan hanya persoalan menjalankan banyak task sekaligus. Concurrency juga menyangkut urutan eksekusi yang tidak selalu dapat diprediksi, kebutuhan akan mutual exclusion untuk menjaga correctness, serta kemampuan sistem untuk tetap produktif ketika sebagian task sedang menunggu. Dengan demikian, praktikum ini memberikan landasan konseptual yang kuat untuk topik-topik lanjutan seperti semaphore, condition variable, deadlock, starvation, dan pola sinkronisasi yang lebih kompleks.